Министерство образования Республики Беларусь

Учреждения образования

«Брестский государственный технический университет

Факультет электронно-информационных систем

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-12

Выполнил
К. В. Ковалюк,
студент группы ПО-12

Проверил
А. Д. Кулик,
преп. стаж. кафедры ИИТ,
«___» _____2026 г.

Брест 2026

Цель работы: Приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Общее задание:

• Прочитать задания, взятые из каждой группы, соответствующей одному из трех основных типов паттернов;

• Определить паттерн проектирования, который может использоваться при реализации задания. Пояснить свой выбор;

• Реализовать фрагмент программной системы, используя выбранный паттерн. Реализовать все необходимые дополнительные классы.

Вариант: №7

1) **Первая группа заданий (порождающий паттерн):**

Преподаватель. Класс должен обеспечивать одновременное взаимодействие с несколькими объектами класса Студент. Основные функции преподавателя: ПроверитьЛабораторнуюРаботу, ПровестиКонсультацию, ПринятьЭкзамен, ВыставитьОтметку, ПровестиЛекцию.

Выполнение:

Код программы 1:

```
class Teacher:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.students = []
            cls._instance.name = "Преподаватель Иванов"
        return cls._instance

    def add_student(self, student):
        self.students.append(student)
        print(f"{student.name} добавлен в группу")

    def check_laboratory_work(self, student, lab_name):
        if student in self.students:
            print(f"{self.name}: Проверил лабораторную работу '{lab_name}' у {student.name}")
            student.lab_results[lab_name] = "Проверено"
        else:
            print(f"Студент {student.name} не числится в группе")

    def conduct_consultation(self, topic):
        print(f"{self.name}: Провёл консультацию по теме '{topic}'")
        for student in self.students:
            print(f"  - {student.name} присутствовал на консультации")

    def take_exam(self, student, exam_name):
        if student in self.students:
            print(f"{self.name}: Принял экзамен '{exam_name}' у {student.name}")
        else:
            print(f"Студент {student.name} не числится в группе")

    def set_grade(self, student, subject, grade):
        if student in self.students:
            student.grades[subject] = grade
            print(f"{self.name}: Выставил отметку {grade} студенту {student.name} по предмету '{subject}'")
        else:
            print(f"Студент {student.name} не числится в группе")

    def conduct_lecture(self, topic):
        print(f"{self.name}: Провёл лекцию на тему '{topic}'")
        for student in self.students:
```

```
        print(f"  - {student.name} прослушал лекцию")


class Student:
    def __init__(self, name):
        self.name = name
        self.grades = {}
        self.lab_results = {}


if __name__ == "__main__":
    teacher1 = Teacher()
    teacher2 = Teacher()

    print(f"teacher1 is teacher2: {teacher1 is teacher2}")
    print(f"Имя преподавателя: {teacher1.name}")

    student1 = Student("Анна Петрова")
    student2 = Student("Иван Сидоров")
    student3 = Student("Мария Иванова")

    teacher1.add_student(student1)
    teacher1.add_student(student2)
    teacher1.add_student(student3)

    print("\n--- Проведение лекции ---")
    teacher1.conduct_lecture("Основы программирования")

    print("\n--- Проверка лабораторных работ ---")
    teacher1.check_laboratory_work(student1, "Лабораторная работа №1")
    teacher1.check_laboratory_work(student2, "Лабораторная работа №2")

    print("\n--- Консультация ---")
    teacher1.conduct_consultation("Паттерны проектирования")

    print("\n--- Приём экзамена ---")
    teacher1.take_exam(student1, "Экзамен по Python")
    teacher1.take_exam(student3, "Экзамен по Python")

    print("\n--- Выставление оценок ---")
    teacher1.set_grade(student1, "Программирование", 5)
    teacher1.set_grade(student2, "Программирование", 4)
    teacher1.set_grade(student3, "Программирование", 5)

    print("\n--- Итоговые данные студентов ---")
    for student in [student1, student2, student3]:
        print(f"Студент: {student.name}, Оценки: {student.grades}")
```

Результат работы программы:

```
teacher1 is teacher2: True
Имя преподавателя: Преподаватель Иванов
Анна Петрова добавлен в группу
Иван Сидоров добавлен в группу
Мария Иванова добавлен в группу

--- Проведение лекции ---
Преподаватель Иванов: Провёл лекцию на тему 'Основы программирования'
  - Анна Петрова прослушал лекцию
  - Иван Сидоров прослушал лекцию
  - Мария Иванова прослушал лекцию

--- Проверка лабораторных работ ---
Преподаватель Иванов: Проверил лабораторную работу 'Лабораторная работа №1' у Анна Петрова
Преподаватель Иванов: Проверил лабораторную работу 'Лабораторная работа №2' у Иван Сидоров
```

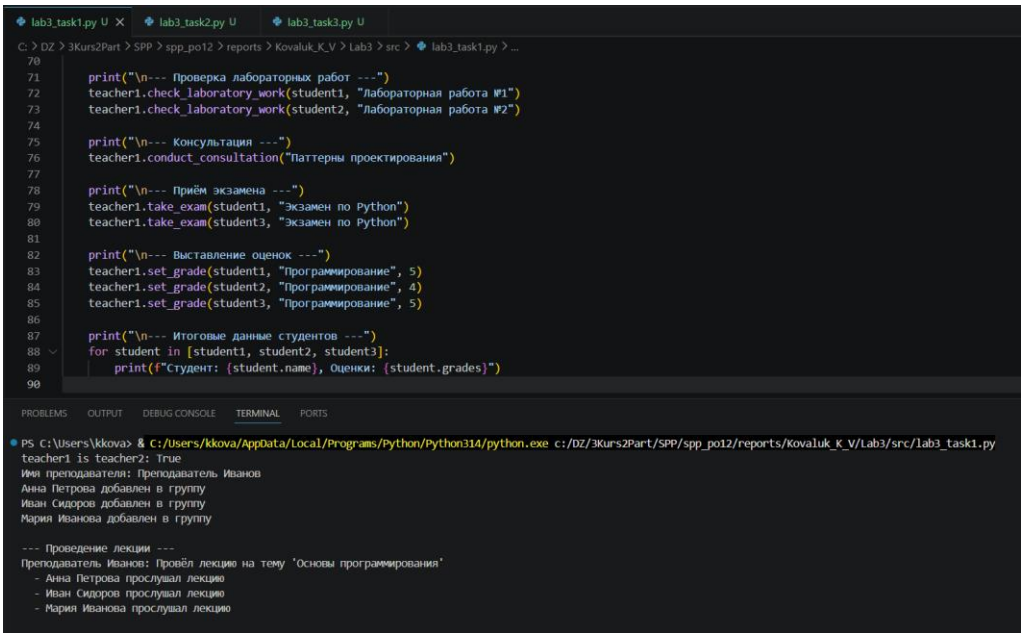
--- Консультация ---
Преподаватель Иванов: Провёл консультацию по теме 'Паттерны проектирования'
- Анна Петрова присутствовал на консультации
- Иван Сидоров присутствовал на консультации
- Мария Иванова присутствовал на консультации

--- Приём экзамена ---
Преподаватель Иванов: Принял экзамен 'Экзамен по Python' у Анна Петрова
Преподаватель Иванов: Принял экзамен 'Экзамен по Python' у Мария Иванова

--- Выставление оценок ---
Преподаватель Иванов: Выставил отметку 5 студенту Анна Петрова по предмету 'Программирование'
Преподаватель Иванов: Выставил отметку 4 студенту Иван Сидоров по предмету 'Программирование'
Преподаватель Иванов: Выставил отметку 5 студенту Мария Иванова по предмету 'Программирование'

--- Итоговые данные студентов ---
Студент: Анна Петрова, Оценки: {'Программирование': 5}
Студент: Иван Сидоров, Оценки: {'Программирование': 4}
Студент: Мария Иванова, Оценки: {'Программирование': 5}

Рисунок с результатом работы программы:



2) **Вторая группа заданий (структурный паттерн):**

ДУ автомобиля. Реализовать иерархию автомобилей для конкретных производителей и иерархию средств дистанционного управления.

Автомобили должны иметь присущие им атрибуты и функции. ДУ имеет три основные функции: удаленная активация сигнализации, удаленноеоткрытие/закрытие дверей и удаленный запуск двигателя. Эти функции должны отличаться по своим характеристикам для различных устройств ДУ.

Выполнение:

Код программы 2:

```
from abc import ABC, abstractmethod


class RemoteControl(ABC):
    @abstractmethod
    def activate_alarm(self, car_model):
        pass

    @abstractmethod
    def remote_lock_doors(self, car_model):
```

```
        pass

    @abstractmethod
    def remote_start_engine(self, car_model):
        pass


class PremiumRemote(RemoteControl):
    def activate_alarm(self, car_model):
        return f"{car_model}: Сигнализация активирована (дальность 500м, датчик удара, GPS-трекинг)"

    def remote_lock_doors(self, car_model):
        return f"{car_model}: Двери заблокированы + автоматическое закрытие окон и складывание зеркал"

    def remote_start_engine(self, car_model):
        return f"{car_model}: Двигатель запущен (автозапуск, климат-контроль, подогрев сидений)"


class StandardRemote(RemoteControl):
    def activate_alarm(self, car_model):
        return f"{car_model}: Сигнализация активирована (дальность 100м, звуковой сигнал)"

    def remote_lock_doors(self, car_model):
        return f"{car_model}: Двери заблокированы"

    def remote_start_engine(self, car_model):
        return f"{car_model}: Двигатель запущен (требуется нажатие кнопки в зоне видимости)"


class BasicRemote(RemoteControl):
    def activate_alarm(self, car_model):
        return f"{car_model}: Сигнализация активирована (дальность 30м)"

    def remote_lock_doors(self, car_model):
        return f"{car_model}: Двери заблокированы (только центральный замок)"

    def remote_start_engine(self, car_model):
        return f"{car_model}: Запуск двигателя не поддерживается данным пультом"


class Car:
    def __init__(self, model, remote):
        self.model = model
        self._remote = remote
        self.engine_running = False
        self.doors_locked = False
        self.alarm_active = False

    def activate_alarm(self):
        result = self._remote.activate_alarm(self.model)
        self.alarm_active = True
        return result

    def lock_doors(self):
        result = self._remote.remote_lock_doors(self.model)
        self.doors_locked = True
        return result

    def unlock_doors(self):
        self.doors_locked = False
        return f"{self.model}: Двери разблокированы"

    def start_engine(self):
        result = self._remote.remote_start_engine(self.model)
        if "запущен" in result or "Запущен" in result:
            self.engine_running = True
```

```
        return result

    def stop_engine(self):
        self.engine_running = False
        return f"{self.model}: Двигатель остановлен"

    def get_status(self):
        return {
            "model": self.model,
            "engine": "работает" if self.engine_running else "выключен",
            "doors": "закрыты" if self.doors_locked else "открыты",
            "alarm": "активна" if self.alarm_active else "выключена"
        }


class BMW(Car):
    def __init__(self, model, remote):
        super().__init__(f"BMW {model}", remote)
        self.manufacturer = "BMW"
        self.has_comfort_access = True


class Tesla(Car):
    def __init__(self, model, remote):
        super().__init__(f"Tesla {model}", remote)
        self.manufacturer = "Tesla"
        self.has_electric_mode = True


class Toyota(Car):
    def __init__(self, model, remote):
        super().__init__(f"Toyota {model}", remote)
        self.manufacturer = "Toyota"
        self.has_economy_mode = True


if __name__ == "__main__":
    print("--- Комбинация 1: BMW X5 + Премиум пульт ---")
    bmw = BMW("X5", PremiumRemote())
    print(bmw.activate_alarm())
    print(bmw.lock_doors())
    print(bmw.start_engine())
    print(bmw.get_status())

    print("\n--- Комбинация 2: Tesla Model 3 + Стандартный пульт ---")
    tesla = Tesla("Model 3", StandardRemote())
    print(tesla.activate_alarm())
    print(tesla.lock_doors())
    print(tesla.start_engine())
    print(tesla.get_status())

    print("\n--- Комбинация 3: Toyota Camry + Базовый пульт ---")
    toyota = Toyota("Camry", BasicRemote())
    print(toyota.activate_alarm())
    print(toyota.lock_doors())
    print(toyota.start_engine())
    print(toyota.get_status())

    print("\n--- Смена пульта на лету ---")
    print("Toyota теперь получает премиум пульт:")
    toyota._remote = PremiumRemote()
    print(toyota.start_engine())
    print(toyota.get_status())
```

Результат работы программы:

```
--- Комбинация 1: BMW X5 + Премиум пульт ---
```

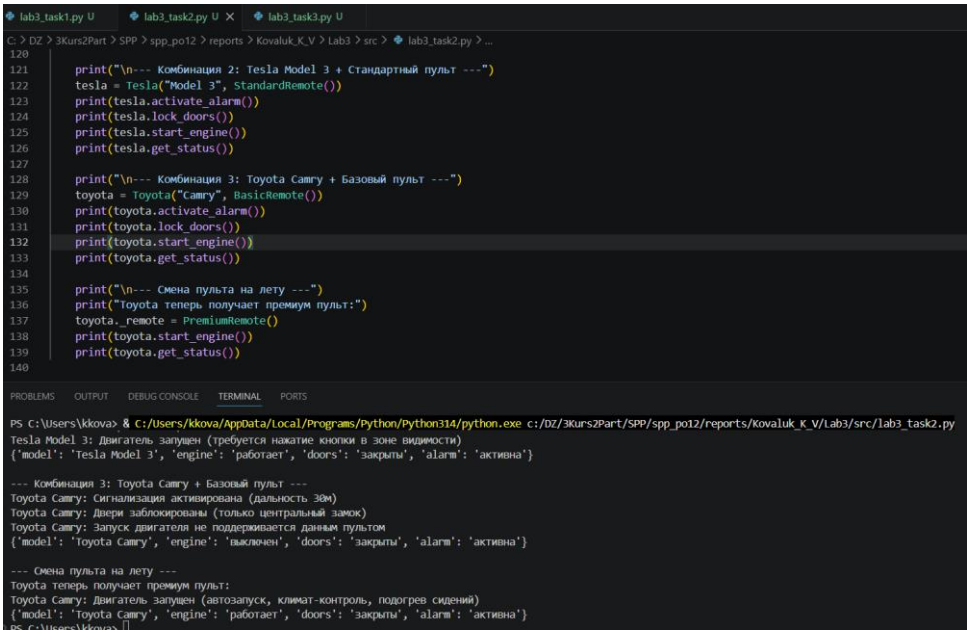
BMW X5: Сигнализация активирована (дальность 500м, датчик удара, GPS-трекинг)
BMW X5: Двери заблокированы + автоматическое закрытие окон и складывание зеркал
BMW X5: Двигатель запущен (автозапуск, климат-контроль, подогрев сидений)
{'model': 'BMW X5', 'engine': 'работает', 'doors': 'закрыты', 'alarm': 'активна'}

--- Комбинация 2: Tesla Model 3 + Стандартный пульт ---
Tesla Model 3: Сигнализация активирована (дальность 100м, звуковой сигнал)
Tesla Model 3: Двери заблокированы
Tesla Model 3: Двигатель запущен (требуется нажатие кнопки в зоне видимости)
{'model': 'Tesla Model 3', 'engine': 'работает', 'doors': 'закрыты', 'alarm': 'активна'}

--- Комбинация 3: Toyota Camry + Базовый пульт ---
Toyota Camry: Сигнализация активирована (дальность 30м)
Toyota Camry: Двери заблокированы (только центральный замок)
Toyota Camry: Запуск двигателя не поддерживается данным пультом
{'model': 'Toyota Camry', 'engine': 'выключен', 'doors': 'закрыты', 'alarm': 'активна'}

--- Смена пульта на лету ---
Toyota теперь получает премиум пульт:
Toyota Camry: Двигатель запущен (автозапуск, климат-контроль, подогрев сидений)
{'model': 'Toyota Camry', 'engine': 'работает', 'doors': 'закрыты', 'alarm': 'активна'}

Рисунок с результатом работы программы:



3) **Третья группа заданий (поведенческий паттерн):**

Проект «Пиццерия». Реализовать формирование заказ(а)ов, их отмену, а также повторный заказ с теми же самыми позициями.

Выполнение:

Код программы 3:

```
from copy import deepcopy


class Pizza:
    def __init__(self, name, size, price, ingredients=None):
        self.name = name
        self.size = size
        self.price = price
        self.ingredients = ingredients or []

    def __str__(self):
        return f"{self.name} ({self.size}) - {self.price} руб"


class Order:
```

```
    def __init__(self, order_id=None):
        self.order_id = order_id
        self.items = []
        self.status = "создан"

    def add_pizza(self, pizza):
        self.items.append(pizza)

    def remove_pizza(self, index):
        if 0 <= index < len(self.items):
            return self.items.pop(index)
        return None

    def total_cost(self):
        return sum(pizza.price for pizza in self.items)

    def get_items_description(self):
        return [str(pizza) for pizza in self.items]

    def __str__(self):
        items_str = ", ".join([pizza.name for pizza in self.items])
        return f"Заказ #{self.order_id}: {items_str} - Итого: {self.total_cost()} руб"


class Command:
    def execute(self):
        pass

    def undo(self):
        pass


class CreateOrderCommand(Command):
    def __init__(self, pizzeria, pizzas, order_id=None):
        self.pizzeria = pizzeria
        self.pizzas = pizzas
        self.order_id = order_id
        self.created_order = None

    def execute(self):
        self.created_order = self.pizzeria.create_order(self.pizzas, self.order_id)
        print(f"{self.created_order}")
        return self.created_order

    def undo(self):
        if self.created_order:
            self.pizzeria.cancel_order(self.created_order.order_id)
            print(f"Отменён {self.created_order}")


class CancelOrderCommand(Command):
    def __init__(self, pizzeria, order_id):
        self.pizzeria = pizzeria
        self.order_id = order_id
        self.cancelled_order = None

    def execute(self):
        self.cancelled_order = self.pizzeria.cancel_order(self.order_id)
        if self.cancelled_order:
            print(f"Отменён {self.cancelled_order}")
        else:
            print(f"Заказ #{self.order_id} не найден")
        return self.cancelled_order

    def undo(self):
        if self.cancelled_order:
```

```
        restored = self.pizzeria.create_order(
            self.cancelled_order.items,
            self.cancelled_order.order_id
        )
        print(f"Восстановлен {restored}")


class RepeatLastOrderCommand(Command):
    def __init__(self, pizzeria):
        self.pizzeria = pizzeria
        self.repeated_order = None
        self.original_order = None

    def execute(self):
        self.original_order = self.pizzeria.get_last_order()
        if self.original_order:
            copied_pizzas = deepcopy(self.original_order.items)
            self.repeated_order = self.pizzeria.create_order(copied_pizzas)
            print(f"Повторный {self.repeated_order} (копия заказа #{self.original_order.order_id})")
        else:
            print("Нет предыдущих заказов для повторения")
        return self.repeated_order

    def undo(self):
        if self.repeated_order:
            self.pizzeria.cancel_order(self.repeated_order.order_id)
            print(f"Отменён повторный {self.repeated_order}")


class AddPizzaCommand(Command):
    def __init__(self, pizzeria, order_id, pizza):
        self.pizzeria = pizzeria
        self.order_id = order_id
        self.pizza = pizza
        self.added = False

    def execute(self):
        self.added = self.pizzeria.add_pizza_to_order(self.order_id, self.pizza)
        if self.added:
            print(f"Добавлена {self.pizza} в заказ #{self.order_id}")
        else:
            print(f"Заказ #{self.order_id} не найден")
        return self.added

    def undo(self):
        if self.added:
            self.pizzeria.remove_last_pizza(self.order_id)
            print(f"Удалена последняя пицца из заказа #{self.order_id}")


class Pizzeria:
    def __init__(self):
        self.orders = {}
        self.next_order_id = 1
        self.order_history = []

    def create_order(self, pizzas, custom_id=None):
        order_id = custom_id if custom_id else self.next_order_id
        if not custom_id:
            self.next_order_id += 1

        order = Order(order_id)
        for pizza in pizzas:
            order.add_pizza(pizza)

        self.orders[order_id] = order
```

```
        return order

    def cancel_order(self, order_id):
        if order_id in self.orders:
            cancelled = self.orders.pop(order_id)
            cancelled.status = "отменён"
            return cancelled
        return None

    def get_order(self, order_id):
        return self.orders.get(order_id)

    def get_last_order(self):
        if self.orders:
            last_id = max(self.orders.keys())
            return self.orders[last_id]
        return None

    def add_pizza_to_order(self, order_id, pizza):
        order = self.get_order(order_id)
        if order:
            order.add_pizza(pizza)
            return True
        return False

    def remove_last_pizza(self, order_id):
        order = self.get_order(order_id)
        if order and order.items:
            return order.remove_pizza(-1)
        return None

    def show_all_orders(self):
        print("\n--- Текущие заказы в пиццерии ---")
        if not self.orders:
            print("Нет активных заказов")
        else:
            for order in self.orders.values():
                print(f"  {order}")


class CommandInvoker:
    def __init__(self):
        self.history = []

    def execute_command(self, command):
        result = command.execute()
        self.history.append(command)
        return result

    def undo_last(self):
        if self.history:
            command = self.history.pop()
            command.undo()
        else:
            print("Нет команд для отмены")


if __name__ == "__main__":
    pizzeria = Pizzeria()
    invoker = CommandInvoker()

    margherita = Pizza("Маргарита", "30см", 450)
    pepperoni = Pizza("Пепперони", "35см", 580)
    quattro_formaggi = Pizza("Четыре сыра", "30см", 520)
    hawaiian = Pizza("Гавайская", "35см", 550)
```

```
    print("\n--- 1. Создание первого заказа ---")
    cmd1 = CreateOrderCommand(pizzeria, [margherita, pepperoni])
    invoker.execute_command(cmd1)
    pizzeria.show_all_orders()

    print("\n--- 2. Добавление пиццы в заказ ---")
    cmd2 = AddPizzaCommand(pizzeria, 1, quattro_formaggi)
    invoker.execute_command(cmd2)
    order = pizzeria.get_order(1)
    print(f"Заказ #{order.order_id}: {order.get_items_description()}")
    print(f"Общая стоимость: {order.total_cost()} руб")

    print("\n--- 3. Создание второго заказа ---")
    cmd3 = CreateOrderCommand(pizzeria, [hawaiian])
    invoker.execute_command(cmd3)
    pizzeria.show_all_orders()

    print("\n--- 4. Отмена последней команды (добавление пиццы) ---")
    invoker.undo_last()
    order = pizzeria.get_order(1)
    print(f"После отмены: заказ #{order.order_id}: {order.get_items_description()}")
    print(f"Стоимость: {order.total_cost()} руб")

    print("\n--- 5. Повтор последнего заказа ---")
    cmd4 = RepeatLastOrderCommand(pizzeria)
    invoker.execute_command(cmd4)
    pizzeria.show_all_orders()

    print("\n--- 6. Отмена заказа через команду ---")
    cmd5 = CancelOrderCommand(pizzeria, 2)
    invoker.execute_command(cmd5)
    pizzeria.show_all_orders()

    print("\n--- 7. Восстановление отменённого заказа (undo) ---")
    invoker.undo_last()
    pizzeria.show_all_orders()

    print("\n--- 8. Итоговая информация ---")
    print(f"Всего заказов в системе: {len(pizzeria.orders)}")
    for order in pizzeria.orders.values():
        print(f"  {order}")
        print(f"    Статус: {order.status}")
        print(f"    Позиции: {order.get_items_description()}")
```

Результат работы программы:

```
--- 1. Создание первого заказа ---
Заказ #1: Маргарита, Пепперони - Итого: 1030 руб

--- Текущие заказы в пиццерии ---
  Заказ #1: Маргарита, Пепперони - Итого: 1030 руб

--- 2. Добавление пиццы в заказ ---
Добавлена Четыре сыра (30см) - 520 руб в заказ #1
Заказ #1: ['Маргарита (30см) - 450 руб', 'Пепперони (35см) - 580 руб', 'Четыре сыра (30см) - 520 руб']
Общая стоимость: 1550 руб

--- 3. Создание второго заказа ---
Заказ #2: Гавайская - Итого: 550 руб

--- Текущие заказы в пиццерии ---
  Заказ #1: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб
  Заказ #2: Гавайская - Итого: 550 руб

--- 4. Отмена последней команды (добавление пиццы) ---
Отменён Заказ #2: Гавайская - Итого: 550 руб
```

После отмены: заказ #1: ['Маргарита (30см) - 450 руб', 'Пепперони (35см) - 580 руб', 'Четыре сыра (30см) - 520 руб']
Стоимость: 1550 руб

--- 5. Повтор последнего заказа ---
Повторный Заказ #3: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб (копия заказа #1)

--- Текущие заказы в пиццерии ---
Заказ #1: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб
Заказ #3: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб

--- 6. Отмена заказа через команду ---
Заказ #2 не найден

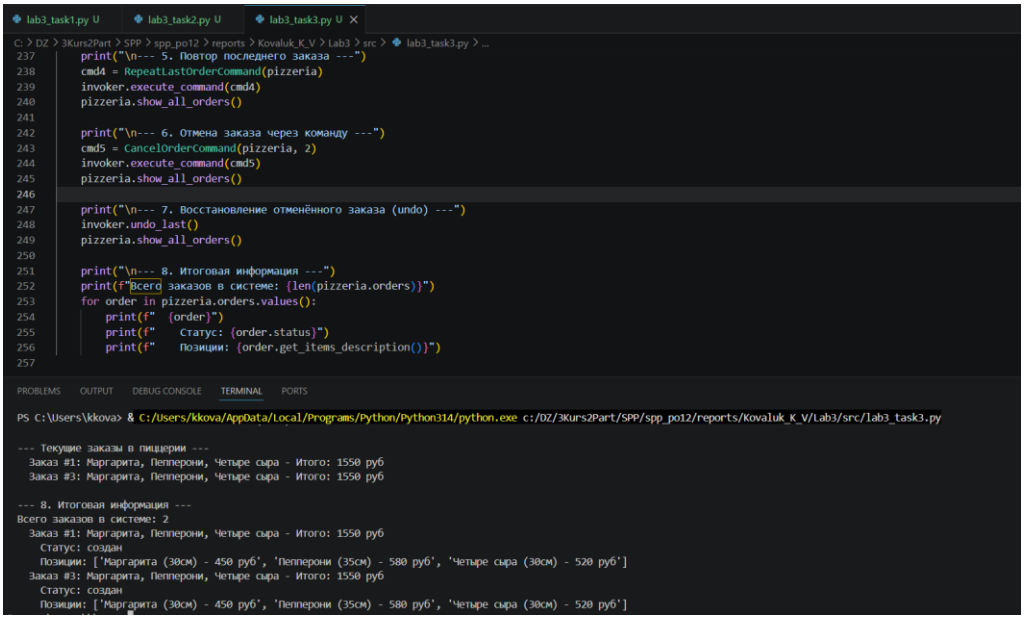
--- Текущие заказы в пиццерии ---
Заказ #1: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб
Заказ #3: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб

--- 7. Восстановление отменённого заказа (undo) ---

--- Текущие заказы в пиццерии ---
Заказ #1: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб
Заказ #3: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб

--- 8. Итоговая информация ---
Всего заказов в системе: 2
Заказ #1: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб
Статус: создан
Позиции: ['Маргарита (30см) - 450 руб', 'Пепперони (35см) - 580 руб', 'Четыре сыра (30см) - 520 руб']
Заказ #3: Маргарита, Пепперони, Четыре сыра - Итого: 1550 руб
Статус: создан
Позиции: ['Маргарита (30см) - 450 руб', 'Пепперони (35см) - 580 руб', 'Четыре сыра (30см) - 520 руб']

Рисунок с результатом работы программы:



Вывод: В ходе работы я закрепил навыки применения паттернов проектирования при решении практических задач с использованием языка Python.